



Proyecto Semana 08: API con Arquitectura MVC/Capas



Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]



IMPORTANTE: Cada aprendiz trabaja sobre un dominio diferente.



Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que NO está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Entity	Item	{YourEntity}
DTO	ItemDTO	{YourEntity}DTO
Mapper	ItemMapper	{YourEntity}Mapper



Objetivo

Implementar una **arquitectura MVC/Capas completa**:

- DTOs para transferencia de datos
- Mappers para conversión entre capas
- Exception handlers centralizados
- Separación clara de responsabilidades



Requisitos Funcionales (Adapta a tu Dominio)

DTOs (Data Transfer Objects)

```
# Ejemplo genérico (Warehouse)
class ItemDTO(BaseModel):
    """DTO público - sin datos internos"""
    id: int
    sku: str
    name: str
    quantity: int
    zone_name: str # Denormalizado para el cliente

class ItemCreateDTO(BaseModel):
    """DTO para creación"""
    sku: str
    name: str
    quantity: int
    zone_id: int

class ItemInternalDTO(BaseModel):
    """DTO interno - incluye todo"""
    id: int
    sku: str
    name: str
    quantity: int
```

```

zone_id: int
created_by: str # Dato interno
created_at: datetime

```

Mappers

```

# Ejemplo genérico
class ItemMapper:
    @staticmethod
    def to_dto(entity: Item) -> ItemDTO:
        return ItemDTO(
            id=entity.id,
            sku=entity.sku,
            name=entity.name,
            quantity=entity.quantity,
            zone_name=entity.zone.name # Join
        )

    @staticmethod
    def to_entity(dto: ItemCreateDTO, user: str) -> Item:
        return Item(
            sku=dto.sku,
            name=dto.name,
            quantity=dto.quantity,
            zone_id=dto.zone_id,
            created_by=user
        )

    @staticmethod
    def to_dto_list(entities: list[Item]) -> list[ItemDTO]:
        return [ItemMapper.to_dto(e) for e in entities]

```

Exception Handlers

```

# Ejemplo genérico
class ItemNotFoundError(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id
        super().__init__(f"Item {item_id} not found")

class DuplicateSKUError(Exception):
    def __init__(self, sku: str):
        self.sku = sku
        super().__init__(f"SKU {sku} already exists")

@app.exception_handler(ItemNotFoundError)
async def item_not_found_handler(request: Request, exc: ItemNotFoundError):
    return JSONResponse(
        status_code=404,
        content={"error": "item_not_found", "item_id": exc.item_id}
    )

```

Layer Flow

```
Request → Router → Service → Repository → DB
      ↓
      Mapper
      ↓
Response ← DTO ← Service
```

Estructura del Proyecto

```
starter/
├─ main.py
├─ api/
│  ├─ routers/
│  ├─ dtos/
│  └─ mappers/
├─ services/
├─ repositories/
├─ domain/
├─ exceptions/
│  ├─ handlers.py
│  └─ errors.py
├─ pyproject.toml
├─ Dockerfile
└─ docker-compose.yml
```

Criterios de Evaluación

Criterio	Puntos
Funcionalidad (40%)	
DTOs bien diseñados	15
Mappers funcionan	15
Exception handlers	10
Adaptación al Dominio (35%)	
DTOs coherentes con negocio	12
Excepciones específicas	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Separación de capas clara	10
Flujo de datos correcto	10
Código limpio	5
Total	100

Política Anticopia

-  **No copies** el ejemplo genérico "ItemDTO/ItemMapper"
 -  **Diseña** DTOs específicos de tu dominio
 -  **Crea** excepciones relevantes para tu negocio
-

Recursos

- [DTO Pattern](#)
 - [FastAPI Exception Handlers](#)
 - [Pool de Dominios](#)
-

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)